UNIT-1 INTRODUCTION TO C-LANGUAGE AND HISTORY

1. C- Language Elements - C Tokens
2. Variable Declarations and Type of constants
3. Data Types - Executable statements
4. General form of C Program
5. Operators - Arithmetic Expressions
6. Formating Numbers In Program Output
7. Simple If ... Nested If ... Example Programs
8. Else If Ladder
9. Switch Case - Decision In Steps In Algorithm with Ex- Prog
10. While Loop, Do While Loop, For Loop with Ex- Prog.

Scanned with OKEN Scanner

* Algorithm

→ It is a step by step procedure (or) set of rules to be followed to solve the given problem.

Characters Of Algorithm

→ There are 4 characters

1] Well-Defined : each step should be defined very clearly and there should be no ambiguity.

2] Finite : an algorithm should have a limited, defined no of steps.

3] Goal-Oriented : the primary purpose of algorithm is to solve a particular given problem.

4] Sequential : steps are written or performed in a specific logical order.

(i) Algorithm For Preparing Tea

Step-1 :- Start

Step-2 :- On the stove and place the bowl on the stove

Step-3 :- Add milk to the bowl

Step-4 :- Add tea powder and sugar to the bowl.

Step-5 :- Let the tea boil

Step-6 :- The tea is ready to serve

Step-7 :- Stop

Scanned with OKEN Scanner

(ii) Making Peanut Butter And Jelly Sandwich

Step-1 :- Start

Step-2 :- Get two slices of bread

Step-3 :- Spread peanut butter on one side of bread slice

Step-4 :- Spread jelly on other side

Step-5 :- Put the two slices of bread together, with peanut butter and jelly sides facing eachother.

Step-6 :- Enjoy your sandwich

Step-7 :- Stop

(iii) Adding Two Numbers

Step-1 :- Start

Step-2 :- Read the values a & b

Step-3 :- Perform addition of a & b numbers

Step-4 :- Print the results

Step-5 :- Stop

(iv) Finding Area Of Circle

Step-1 :- Start

Step-2 :- Read the values of π and r

Step-3 :- Compute the value of $\pi * r * r$

Step-4 :- Display the output

Step-5 :- Stop

Scanned with OKEN Scanner

(v) Finding Area Of Triangle

Step - 1 :- Start

Step - 2 :- Read the values of l and b

Step - 3 :- Compute the value of $l * b$

Step - 4 :- Print the output

Step - 5 :- Stop

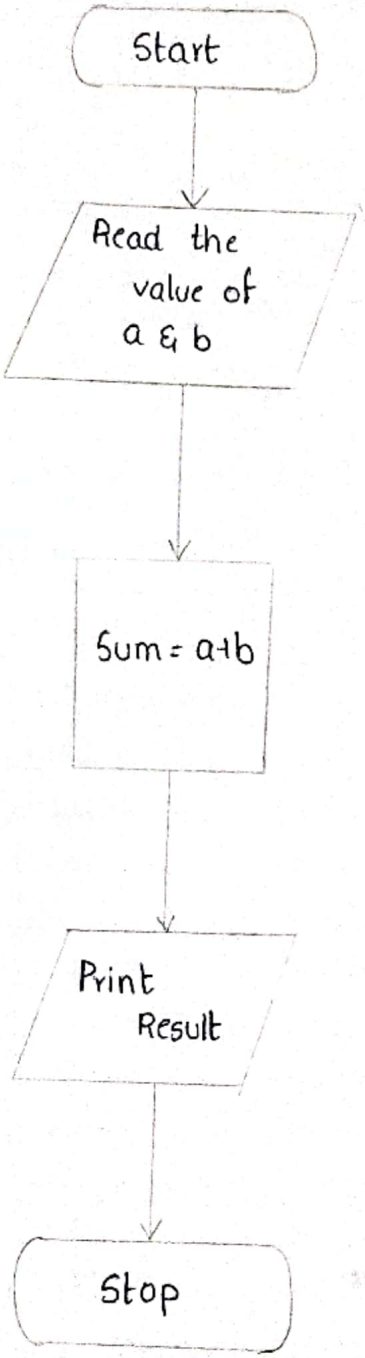
* FLOWCHART

→ It is a pictorial or graphical representation of an algorithm.

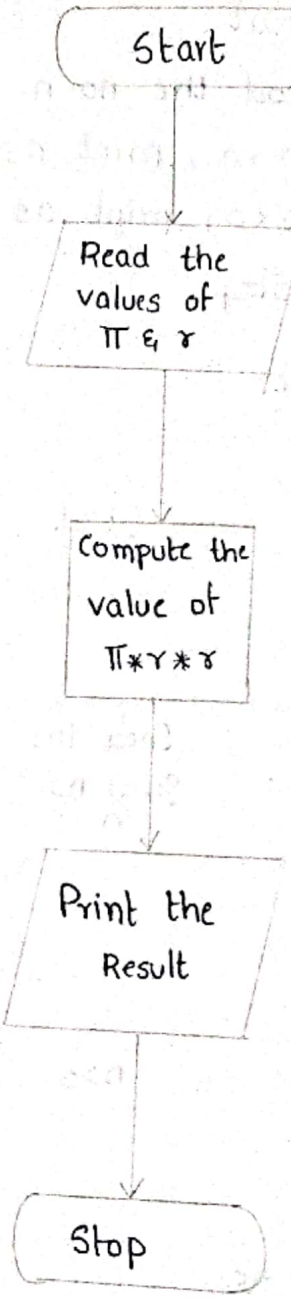
SYMBOL	OPERATION	DESCRIPTION
	Start	→ a start symbol shows the starting & ending point of C-program.
	Process	→ indicates an operation performed by computer such ass add, opp. variable' assign etc.
	Data (input/output)	→ represents input/output of the given problem.
	Decision	→ It is a diamond structure. It always have 2 flow lines, 1 is out and the other is in.
	Flowlines	→ The line represent flow of execution of the given program
	Stop	→ Shows the ending point of the program.

Scanned with OKEN Scanner

(i) Adding Two Numbers



(ii) Area Of Circle



Scanned with OKEN Scanner

9] To check whether the given number is positive or negative

(i) Algorithm

Step-1 :- Start

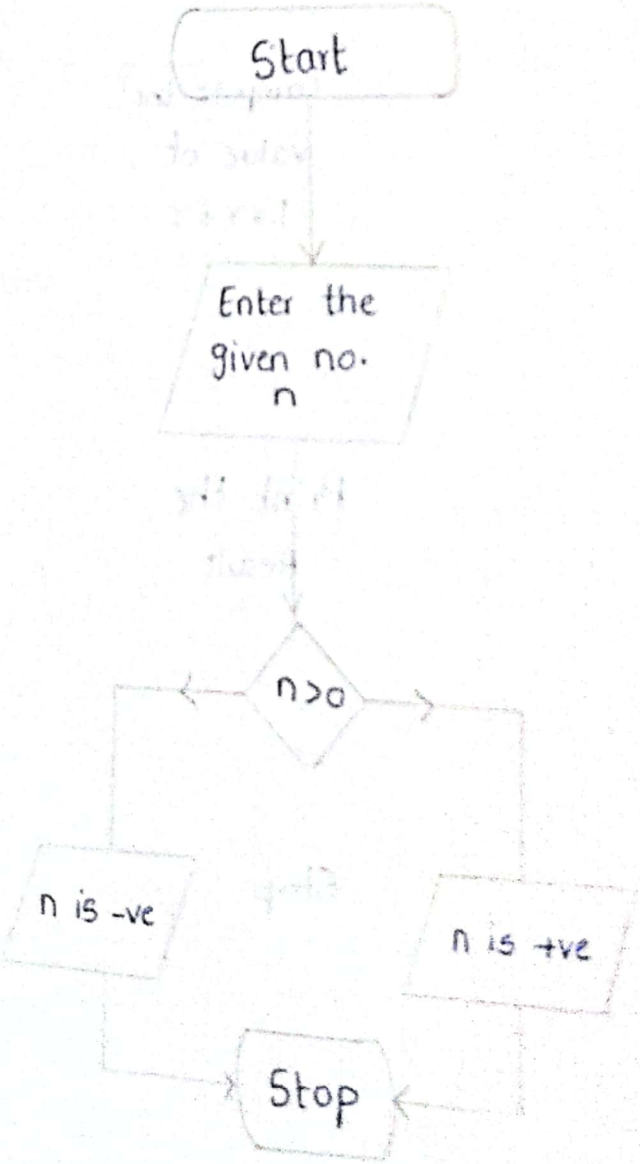
Step-2 :- Read the no. n

Step-3 :- n > 0, print as +ve.

Step-4 :- n < 0, print as negative

step-5 :- Stop

(ii) Flowchart



Scanned with OKEN Scanner

* Elements Of C-Language

→ In C-programming, the language is built from several core elements.

→ These elements form the foundation of any C-programme.

→ In C-programme each and every smallest individual unit is known as token.

→ There are 12 tokens; they are:

1] Character set
2] Identifiers
3] Keywords
4] Constant
5] Variables
6] Data Types
7] Operators
8] Expressions
9] Statements
10] Functions
11] Pre-processor Directives
12] Comments

Character Set

[A-Z → 65-90
a-z → 97-122]

ASCII → American Standard For Information Interchang

→ C programs are written using character form.

→ Letters : A-Z, a-z

→ Digits : 0-9

→ Special Symbols : +, -, *, /, %, =, < >, { }, ( ), [ ]

→ White space characters : space-tab, new line.

Identifiers

→ They are the names given to a variable, array, function etc.

→ These are user defined names and consists of sequence of characters.

→ Example:- a, b, x, y, add, sum, average, total marks

Scanned with OKEN Scanner

Rules

(i) The 1st character must be an alphabet or underscore

(ii) Only letters, digits and underscore shall be used.

(iii) A minimum of 31 characters must be used.

VARIABLES

→ A variable is a container which is used to store the value.

→ The value of the variable changes throughout the execution of the program.

Syntax :- Data Type Variable Name

Ex: int a int a, b;

int a = 10 int a = 10, b = 20;

* PROCEDURE TO RUN A PROGRAM

→ Switch on the computer

→ It will display O.S windows and Ubuntu using down arrow.

→ Select Ubuntu O.S

→ Then it will ask password and user name
↳ aceec ↳ aceec

→ In order to change from aceec to new created folder the command is mkdir 257C2

→ To open a file in text editor to write the program the command is gedit filename.c

→ Then a file will open in new window, where we need to write the program.

Scanned with OKEN Scanner

→ Save the program
→ On the top right side of the screen you will find a save button. to save your program.
→ In order to compile the program, the command is
cc filename.c
→ Upon executing the above command, it will display if there are any errors in the program, if there are no errors, continue with the run command to check the output.
→ To see the output the command is ./a.out.

```
aceec@$: mkdir 257C2↵
aceec@$: cd 257C2↵
aceec@257C2$: gedit hello.c↵
        * hello.c
aceec@257C2$: cc hello.c↵
aceec@257C2$: ./a.out
```

Keywords

→ There are 32 keywords in C-language.
→ Keywords are also known as reserved words.
→ Keywords are the pre-defined words which have specific meaning.
→ Keywords must be written using only lower case letter

Examples

int, float, double, char, switch etc.

Scanned with OKEN Scanner

(i) int int;

→ It is invalid, since keyword shouldn't be used in the case of identifier.

(ii) int INT;

→ It is valid, since it is written in uppercase, which indicates that it is not a keyword.

(iii) float _int;

→ It is not valid, since keyword shouldn't be used.

(iv) int 1_a;

→ It is invalid as it starts with a digit.

(v) int a_1;

→ It is valid as it starts with an alphabet followed by underscore and digit.

(vi) int @a;

→ It is invalid

(vii) int a_@;

→ It is invalid

(viii) int a@;

→ It is invalid

(ix) int a__

→ It is valid

Scanned with OKEN Scanner

OPERATORS

→ In C, operators are the symbols which are used to perform arithmetic operations on one or more operands

→ Operators help us to perform some mathematical, relational bit-wise, conditional and logical computation on values and variables.

→ Values and variables used with operators are called operand

Ex:- a * b

a, b are operands, * operator.

(i) Arithmetic Operators :-

→ C supports arithmetic operations like +, -, *, /, & %

Note:

(1) Division gives the quotient as the result.

(2) Modular gives remainder as the result.

Example

→ For integers a=5, b=5

For real numbers a=14.1, b=5.6

For mixed no.s a=12.2, b=3.

Integers

5+5 = 10

5-5 = 0

5/5 = 1

5*5 = 25

5%5

(ii) Relational Operators

→ Relational operators are also know as relational expressions

→ These are used to calculate relational operations b/w the operands.

→ The value of the relational operations is either 0 or 1

→ 0 indicates false, 1 indicates true.

→ Relational operators are: >, <, >=, <=, ==, !=

(iii) Logical Operators

→ These are used to combine 2 or more relational expressions or operators.

→ There are 3 logical operators:

1) && - logical and
2) || - logical or
3) ! - logical not

→ The value of logical operators is true or false

1) Logical And (&&)

→ Truth table for &&

A	B	A&&B
T	T	T
T	F	F
F	T	F
F	F	F

→ Example: (a>b) && (a>c)

ℓ

Scanned with OKEN Scanner

2] Logical Or (||)

→ Truth table

A	B	A || B
T	T	T
T	F	T
F	T	T
F	F	F

Example : (a<b) || (a>c)

→ It evaluates false when both expressions are false.

→ It evaluates true when one expression is true.

3] Logical Not (!)

→ It evaluates true when expression is false

→ It evaluates false when expression is true.

A	B
T	F
F	T

→ Example: a ! b

Ex: a=10, b=5, c=3

(i) (a>b) && (a>c)
(10>5) && (10>3) } T && T → T

(ii) (b>a) && (b>c)
(5>10) && (5>3)
T && T → F

Ex:- a=2, b=10, c=15

(i) (b>a) || (b>c)

(10>2) || (10>15)

T || F → T

(iv) Assignment Operators

→ These operators are used to assign a value to a variable.

→ Ex:- a=10,

Here we are assigning a value 10 to the variable 10.

(v) Compound Assignment Operators / Short Hand Operators.

→ Combination of marithmetic operators with assignment operator is known as compound assignment operator short hand operator.

→ +=, -=, *=, /=, %=

→ Example: a=10

a = a+2 ⟹ a = 10+2 = 12 **a=12**

a = a-2 ⟹ a = 10-2 = 8 **a=8**

a = a*2 ⟹ a = 10*2 = 20 **a=20**

a = a/2 ⟹ a = 10/2 = 5 **a=5**

a = a%2 ⟹ a = 10%2 = 0 **a=0**

Scanned with OKEN Scanner

(vi) Increment & Decrement Operators

→ Increment operator is ++, which means value is incremented by 1.

→ The decrement operator is --, which means value is decremented by 1.

Example: Increment

a = 20

a++ ⟹ a = a+1

a = 20+1

a = 21

Example: Decrement

a = 15

a-- ⟹ a = a-1

a = 15-1

a = 14.

→ Increment and decrement operators are divided into 2 parts:

(i) pre

(ii) post.

→ pre-increment ⟹ ++a

pre-decrement ⟹ --a

post-increment ⟹ a++

post-decrement ⟹ a--

→ Example:

(i) a=6

++a

⇓ a=6

a=7.

(ii) a=5

b=a++

⇓ a=5

b=6

(iii) a=5

b=++a

⇓

a=6

b=6.

→ In pre-increment, the value will be incremented and then assigned to a given variable.

increment & assigned.

→ Assign & increment

Scanned with OKEN Scanner

Pre-increment

increment & assign

Pre- Decrement

decrement & assign

Post - increment

assign & increment

Post- decrement

assign & decrement

→ Example

i) $a = 6$
$a--;$
$a = a-1$
$= 6-1$
$a = 5$

(ii) $a = 6$
$--a;$
$a = a-1$
$a = 6-1$
$a = 5$

(iii) $a = 5$
$b = a--;$
$b = 5$
$a = 4.$

(iv) $b = --a$
$a = 4$
$b = 4$

(v) $i = 5, j = 6, k;$
$k = i++ + j++$

(vii) Bit- Wise Operators

→ Bit- wise operators are used to manipulate the data at bit level.

→ There are six bit-wise operators

Scanned with OKEN Scanner

OPERATOR	MEANING
&	bit-wise and
|	bit-wise or
^	bit-wise exclusive or
~	negation (compliment)
>>	right shift
<<	left shit

→ 0 - 0 0 0
1 - 0 0 1
2 - 0 1 0
3 - 0 1 1
4 - 1 0 0
5 - 1 0 1
6 - 1 1 0
7 - 1 1 1

Example

(i) Bitwise - And

```
  0 0 1  → 1
  1 0 1  → 5
---------
  1 0 1  → 1
```

(ii) Bitwise - Or

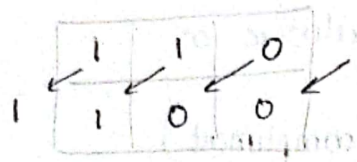
```
  1 1 1  → 7
---------
  1 1 1  → 7
```

Scanned with OKEN Scanner

* Left Shift

→ When we perform the left shift on binary values, the no. will move by left.

Example :- x << 1 , x = 6

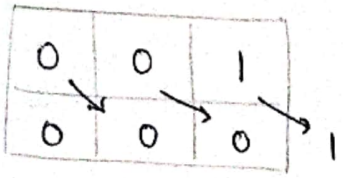


→ The value of x will move 1 digit left.

* Right Shift

→ When we perform right shift on the binary equivalent the number will move by right by 1 digit.

→ Example: x >> 1



(viii) Conditional Operator

→ Ternary operator is a type of operator which is used to determine true or false.

Syntax: Condition ? exp1 : exp2 ;

(→ Ternary operator; → expression)

Example: (a>b) ? exp1 : exp2;

→ If (a>b) is true then exp1 will evaluate

→ If (a>b) is false then exp2 will evaluate.

⇒ a=10, b=5
10>5 ⇒ Evaluate exp1

→ a=4, b=10
(4>10) → Evaluate exp2

Scanned with OKEN Scanner

(ix) Special Operators :

→ C language supports some special operators like size of (), pointers (*), member selection (⟶)

→ Size of operator is used to display the size of a data type.

Syntax :- Sizeof (data type);

Ex : Sizeof (int); ⟶ 2 bytes

Sizeof (float); ⟶ 4 bytes

Scanned with OKEN Scanner

* Formatted Input And Output Functions :

→ Scanf is a formatted input function, which is use to give input at run time.

Syntax : scanf (" format specifier", address of variable);

Example

```
Scanf ("%d ; &a);
```

→ Printf (" is an formatted output function, which is use to get output on the console.

Syntax : printf (" format specifier", variable);

Example

```
printf("%d, a);
```

→ Format specifier for integer number is int ⟶ %d &
for decimal number is float ⟶ %f.

Q] Write a c-program to perform addition of two number in compile time.

→

```
# include <stdio.h>
int main ( )
{
int   x=10, y=20, z;
    z=x+y;
printf ("%d," z);
}
```

Scanned with OKEN Scanner

Run - Time

```
#include <stdio.h>
    int main ( )
{
int x, y, z;
printf ("Enter two values");
scanf ("%d %d", &x, &y);
  z = x+y;
printf ("sum = %d," z);
}
```

Output :

```
Enter 2 values
45
21
Sum = 66
```

LAB

Q] Write a c-program to perform subtraction of two number using compile and run time.

Q] Write a c-program to perform multiplication of two number using compile and run time.

Q] Write a c-program to perform division of two number using compile and run time.

Q] Write a c-program to perform modulo of two numbe in compile and run-time.

Scanned with OKEN Scanner

6] Write a c-program to perform average of three numbers in compile & run-time.

6] Write a c-program to perform simple interest using compile & run-time.

6] Write a c-program to compute circumference of circle.

6] Write a c-program to compute area of Δ^{le}.

→ Write a c-program to compute relational operators.

```
#include < stdio.h >
int main ()
{
int a = 15, b = 6;
printf ("%d\n", a > b);
printf ("%d\n", a < b);
printf ("%d\n", a ≥ b);
printf ("%d\n", a ≤ b);
printf ("%d\n", a == b);
printf ("%d\n", a != b);
```

Scanned with OKEN Scanner

→ Write a c-program to perform increment & decrement operations.

Pre-increment

```
#include <stdio.h>
int main()
{
int a=6;
  ++a;
printf("values of a is %d", a);
}
```

Post-increment

```
#include<stdio.h>
int main()
{
int a=5,b;
 b= a++;
printf("value of a is %d\n",a);
printf("value of b is %d\n",b);
}
```

Post-Decrement

```
#include<stdio.h>
int main()
{
int a=6;
 a--;
printf("value of a is %d\n",a);
}
```

Pre-increment

```
#include<stdio.h>
int main()
{
int a=6;
 --a;
printf("values of a is %d",a);
}
```

Scanned with OKEN Scanner

Post-decrement

```
#include <stdio.h>
int main
{
int a=5, b;
b = a--;
printf ("value of a is %d\n", a);
printf ("value of b is %d\n", b);
}
```

* STATEMENTS

→ Statements are the set of instructions that has to be executed by the system or program.

→ There are few statements in C

1) Null statement
2) Expression statement
3) Compound statement
4) Return statement
5) Conditional / Selection / Control

NULL STATEMENT

A statement ending with semi-colon is known as null statement

Ex :- int a;

EXPRESSION STATEMENT

An expression ending with semi-colon and which performs the main task in the program is called expression statement.

Ex: $c = a * b;$

$a = \frac{1}{2} * b * h;$

Scanned with OKEN Scanner

Compound Statement

The block of code which combines group of statements is known as Compound Statement.

Ex :-
```
{
    ≡  → Body of statements / compound of statements
}
```

Return Statement

→ A return statement terminates the function

Syntax : return value;

Ex : return 0;

→ Main function returns a value "0" to the operating system which means the program is successfully executed.

Selection / Control / Conditional · (Control Statements)

→ As a name itself suggests control statements, control the flow of execution in the c-program.

→ These statements are used to determine the order in which the statements has to be executed.

→ Selection statements contains a decision making, to see whether the condition is satisfied or not and to execute the respective statements.

→ Control statements are of 5 types

1] Simple if
2] if-else
3] nested-if
4] else-if-ladder
5] switch

Scanned with OKEN Scanner

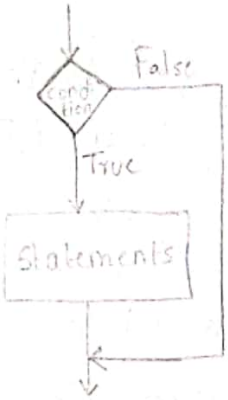
* Simple if (if statement)

If statement, executes only the set of statements when the condition is true.

→ If the condition is false, the control (cursor) directly goes out of the if block, without executing the inside block of statements.

Syntax:
```
if (condition)
{
    // statement block - I
}
```

→ Flowchart for if



Q) Write a c-program for simple if.

```
#include <stdio.h>
int main()
{
int n=5;
if (n==10)
{
printf ("Both are equal");
}
printf ("\n Given values are not equal);
}
```

Scanned with OKEN Scanner

* If-Else Statements

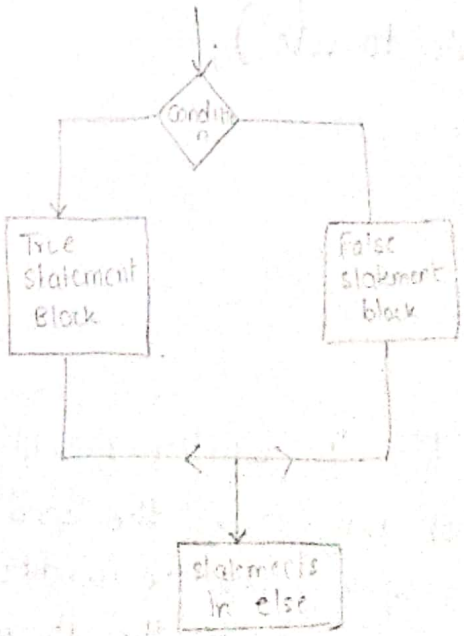
→ In if-else statement, if the condition is true, if-block will execute.

→ If the condition is false, it executes the else block.

↓
It is similar to conditional operator. (?)

Syntax :-
```
if (condition)
{
  ≡
}
Else
{
  ≡
}
```

→ Flowchart for if-else



Scanned with OKEN Scanner

Q) Write a c-program for if-else.

```
#include<stdio.h>
int main()
{
int age;
printf("Enter the age of person);
scanf("%d", &age);
if (age >= 18)
{
printf("person is eligible to vote");
}
else
{
printf("not eligible to vote");
}
return 0;
}
```

* NESTED - if

→ In the nested-if block, first the condition-1 will be chec if it is true, the control again checks the condition to, if it is true, the control again checks the condition 3, if is also true, then the statements inside the if will be executed.

→ If either of the conditions is false, then statements inside the if block will not be executed.

→ If the condition is false, the control comes out of the if block and executes other statements.

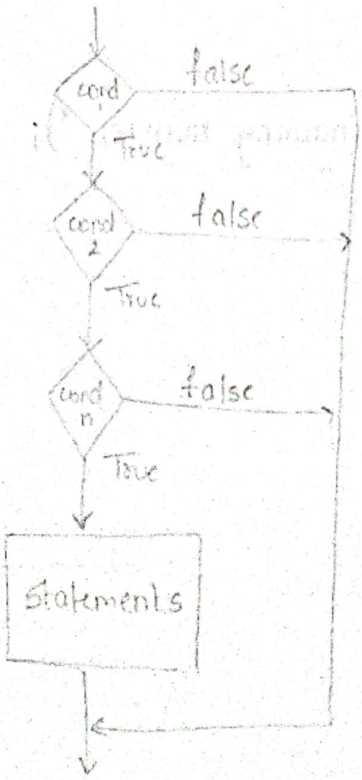
Scanned with OKEN Scanner

Syntax :-

```
if (condition 1)
{
  ≡
if (condition 2)
{
  ≡
if (condition 3)
{
  ≡
}
}
}
```

```
if (condition 1)
{
if (condition 2)
{
if (condition 3)
{
  ≡
}
}
}
```

Flowchart :-



Scanned with OKEN Scanner

8] Write a c-program to check whether the 1st number is greater than the remaining numbers.

```
#include <stdio.h>
int main ()
{
int a,b,c,d;
printf ("Enter the values for a,b,c and d");
scanf ("%d %d %d %d", &a, &b, &c, &d);
if (a>b)  // condition 1
{
if (a>c)  // condition 2
{
if (a>d)  // condition 3
{
printf ("a is greater than remaining numbers");
}  // 1st if close
}  // 2nd if close
}  // 3rd if close
return 0;
}  // main close
```

Scanned with OKEN Scanner

* ELSE - if - LADDER

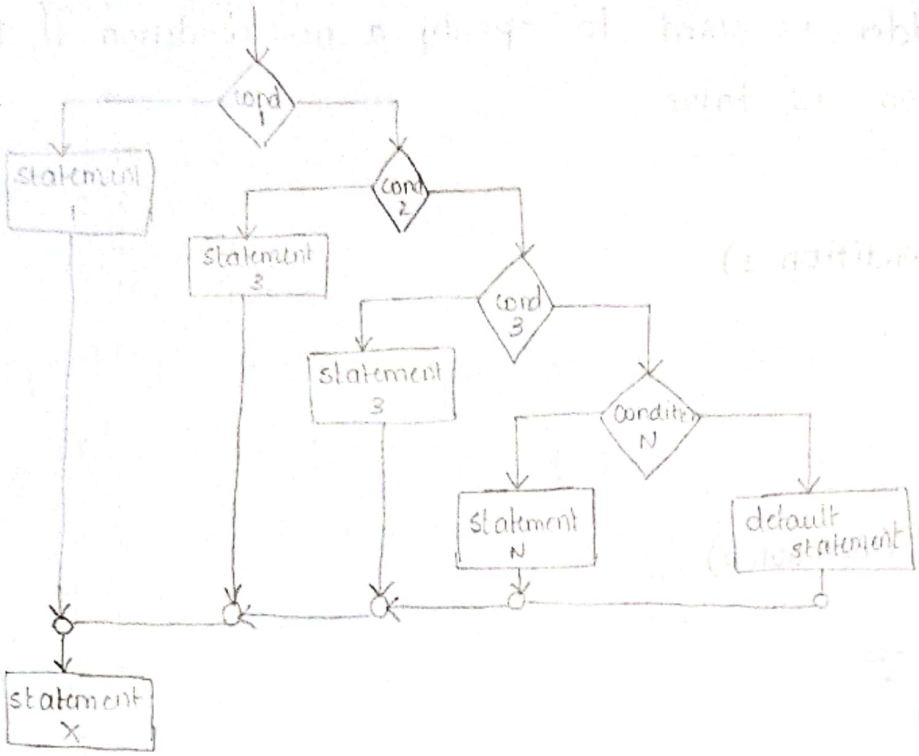
Else - if - ladder is used to specify a new condition if the first condition is false.

Syntax :-

```
if (condition 1)
{
  ≡
}
else
    if (condition 2)
    {
      ≡
    }
else
    if (condition 3)
    {
      ≡
    }
else
  {
    ≡
  }
```

Scanned with OKEN Scanner

Flowchart :



Q) Write a c-program to check whether the given number is +ve or -ve. (if else)

8] Write a c-program to display the grades of a student

```
#include < stdio.h >
int main()
{
  int marks;
  printf ("Enter the marks");
  scanf ("%d", &marks);
  if ("marks ≥ 75 && marks ≤ 100);
  {
  printf ("A Grade");
  }
  else
  if ("marks ≥ 60 && marks < 75);
  {
   printf ("B Grade");
  }
  else
  if ("marks ≥ 50 && marks < 60);
  {
  printf ("C Grade");
  }
  else
  printf ("FAIL");
  }
  return 0;
  }
```

* Switch Statement

When one of the many alternatives is the selected, we can use the if statement to control the selection. However, the complexity of such program increases dramatically when the number of alternatives increases, then the program becomes difficult to read and follow.

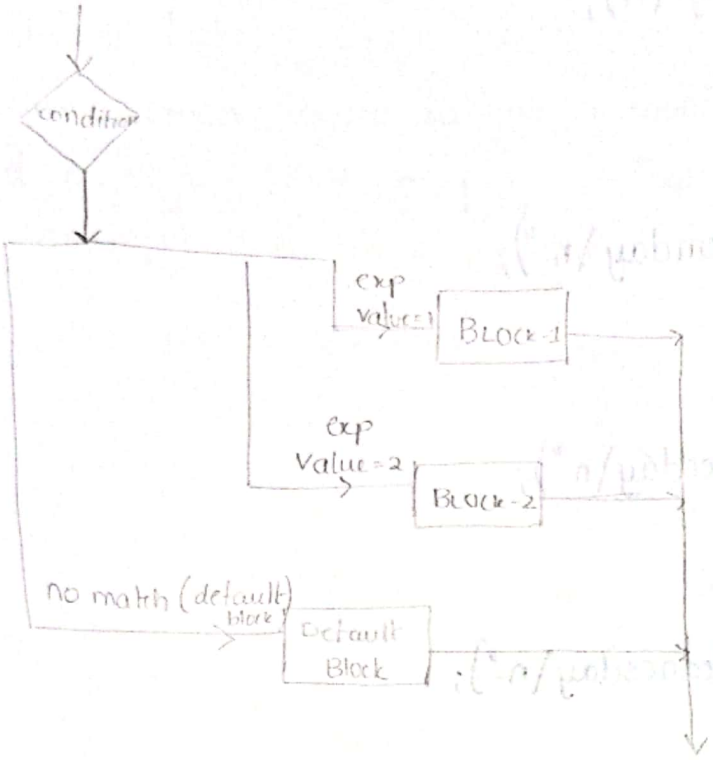
→ C has a built-in multi-way decision statement known as switch.

→ The switch statement test's the value of variable (expression) against a list of cases.

Syntax:

```
Switch (expression) or (variable)
{
Case- Value-1 :
                statement-1;
                break;
Case Value-2 :
                statement-2;
                break;
    .
    .
    .
default :
            default statement :
```

Flowchart :



RULES

(i) Rules, switch statement should evaluate to integer.

(ii) There should be space b/w case and label.

(iii) Labels should be either integer or character constant.

(iv) There should be break defined in each case statement.

(v) Default statement should be written in switch.

Q] Write a c-program to print days of a week based on the user choice using switch case.

```
#include <stdio.h>
int main()
{
 int day;
printf ("Enter a number (1-7)");
scanf ("%d", &day);
switch (day)
{
```

Scanned with OKEN Scanner

```
Case 1 :
    printf ("sunday \n");
    break ;
Case 2 :
    printf (" Monday \n ");
    break;
Case 3 :
    printf (" Tuesday \n ");
    break;
Case 4 :
    printf (" Wednesday \n ");
    break;
Case-5 :
    printf (" Thursday \n ");
    break;
Case 6 :
    printf (" Friday \n ");
    break;
Case-7 :
    printf (" Saturday \n ");
    break;
default :
    printf (" Invalid statement, Enter from (1-7)");
return 0;
}
```

Scanned with OKEN Scanner

LAB

1] The movie ticket:

A cinema owner decides to give discount on movie tickets based on age. Write a program that takes a person's age as input and determines the ticket price.

```
#include <stdio.h>
int main()
{
int age;
printf("Enter the age of the person);
scanf("%d", &age);
if (age <= 10)
{
printf(" Ticket is $5);
else
{
printf(" Ticket is $10");
}
return 0;
}
```

Scanned with OKEN Scanner

2] Weather Assistant

```
#include<stdio.h>
int main()
{
int temperature;
printf("Enter the temperature");
scanf("%d", &temperature);
if (temperature <= 25)
{
printf("Nice weather to play outside");
else
{
printf("Stay indoors");
}
return 0;
}
```

3] Loan Eligibility

```
#include<stdio.h>
int main() {
int loan;
printf("Enter the loan");
Scanf("%d", &loan);
if (
```

Scanned with OKEN Scanner

4] Simple Calculator

```
#include<stdio.h>
int main()
{
int a, b, c;
char op;
printf("Enter the value of 'a & b');
scanf("%d %d", &a, &b);
switch(op)
 {
Case '+': c = a+b;
          printf("sum = %d", c);
          break;
Case '-': c = a-b
          printf("sub = %d", c);
Case '*': c = a*b
          printf("multiply = %d", c);
Case '/': c = a/b
          printf("divide = %d", c);
Case '%': c = a%b
          print("mod = %d", c);
default:
          printf("Not valid");
return 0;
}
```

* Data Type

→ C language is rich in data types.

→ Data types represents the type of a variable (data)

→ C language supports 3 types of data types:

1] primitive data type.

2] derived data type

3] user-defined data type.

Primitive Data Type

→ Primitive data types are integer (int), floating point (float), character data type (char)

→ Integer data type are stored in 2 bytes.

→ Float data type are stored in 32 bits with 6 digits of precision by default.

↳ [1 byte = 8 bits. 4 bytes = 32 bits]

Note :-

1] The no. of digits stored after the decimal point is called precision.

2] Extention of float data type is double, size is 8 bytes.

3] Extention of double is long double, size is 10 bytes.

4] Character data type can be defined as a single character enclosed in ~~sing~~ (' ').

5] Character data types are usually stored using 1 byte.

Scanned with OKEN Scanner

Derived Data Type

→ These are the extension of basic data types, which represents multiple values in a program.

Example : arrays, functions & pointers.

User-Defined Data Type

→ These are the data types which are created by the user itself, by using the keywords like:

- enum
- typedef, etc.

→ User-defined data types are:

1] structure
2] union
3] enum
4] typedef

* STRUCTURE OF C-PROGRAM

Documentation (or) comment section
Headerfile (or) preprocessor directives
Definition Section
main ( ) { Declaration section Execution section

user-defined function
{
≡
}

Global declaration of variable

Scanned with OKEN Scanner

→

```
#include <stdio.h>          ——→ header file section
#define PI = 3.14
int radius, A;              ——→ Global declaration
int main ()
{
printf ("perform Area of circle");
scanf ("%d", & radius);
  A = PI * r * r;
printf ("%d", A);
}
```

→ main function

* LOOPS [CONTROL STRUCTURES]

→ In c-programming language control structures are also known as repetative or iterative statements / structures, they can also be termed as looping statements.

→ In looping, the sequence of the loops are executed until some conditions for termination of the loop are satisfied.

→ A program loop consists of 2 parts : (i) Body
(ii) Head

→ The head of the statement decides the no. of times for which the instructions present in the body of the loop are to be executed.

→ Based on the position of a condition in a loop, the control structures are classified into 2 types.

1] Entry Control Loop (While)

2] Exit Control Loop (Do-While)

Scanned with OKEN Scanner

1] Entry Control Loop (While)

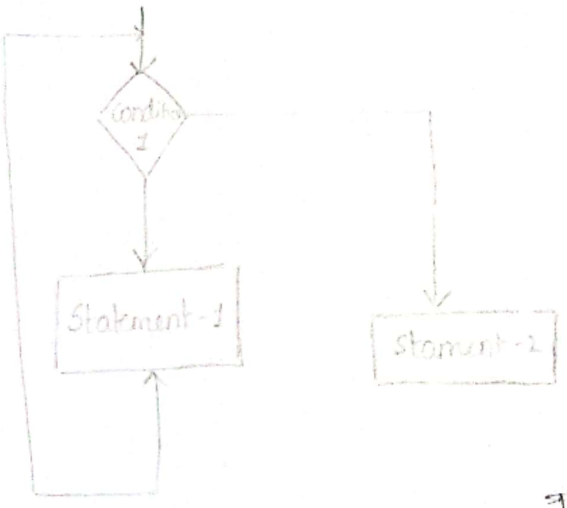
→ In while the test condition is evaluated and if the condition is true then the body of the loop is executed

→ After the execution of the body, the test conditions are again evaluated and if it is true, the body is executed once again.

→ This process will continue till the condition becomes false and the control is transferred to out of the loop.

→ The body of the loop may have 1 or more statements.

Flowchart



Syntax :

```
while (condition)
{
 Statement 1;
 Statement 2;
increment / decrement
}
```

```
#include<stdio.h>
int main()
{
int i=1, n, sum=0;
pf("Enter the value of n");
sf("%d", &n);
while(i<=n)
{
  sum=sum+i;
  i++;
}
pf("Sum of given no's is:
    %d", sum);
}
```

2] Do-While Loop

→ The do-while loop first executes the statements (set of statements) written inside the do loop.

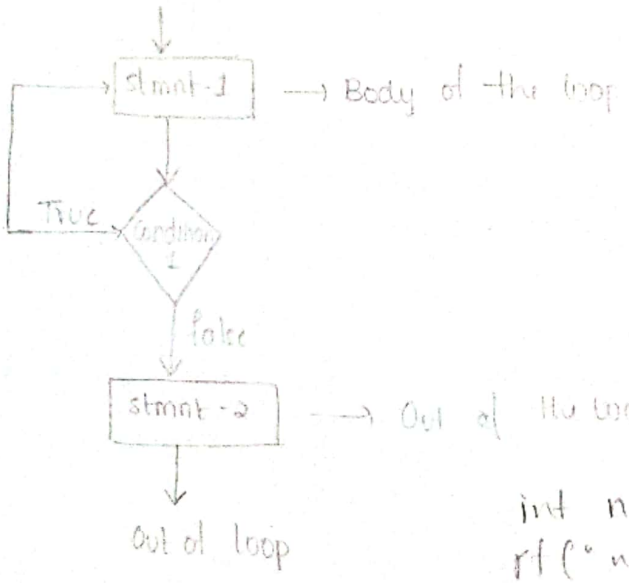
→ Then it will check the condition given in the while.

→ If the condition in the while statement is true, then once again the body of the loop is executed.

→ This process continues till the condition becomes false.

→ Then the loop will be terminated, the control goes to the statement that appears immediately after the while statement

Flowchart



Syntax :

```
do
{
 stmnt 1;
 stmnt 2;
   :
 stmnt n;
increment / decrement
while (condition);
}
```

*] For Loop → Entry Control Loop

→ The for statement / loop is also known as entry control loop.

→ The for statement follows 3 steps

1] Initialisation
2] condition checking
3] Updation (increment / decrement)

→ Initialisation - it sets the loop counter variable to initial value.

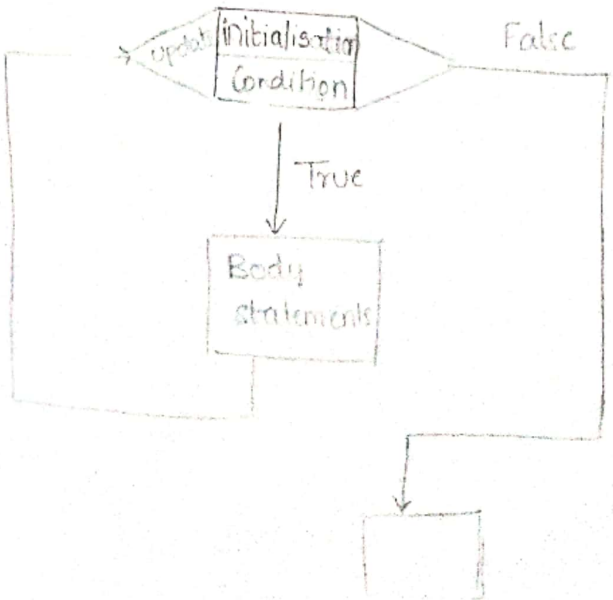
→ Condition checking - to check the condition ($i \leq n$)

→ Updation - after the body of loop is executed, the value of the counter variable is either incremented or decremented and again the condition is tested.

Syntax :

```
for (i=0; i<=n; i++)
{
  ==
  ==
}
```

Flowchart :



Scanned with OKEN Scanner

* Nested For Loop

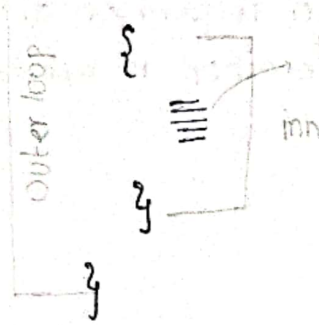
→ In c-language, it is possible to place a loop inside another loop, which is nested loop.

→ Nested Loop contains 2 loop:

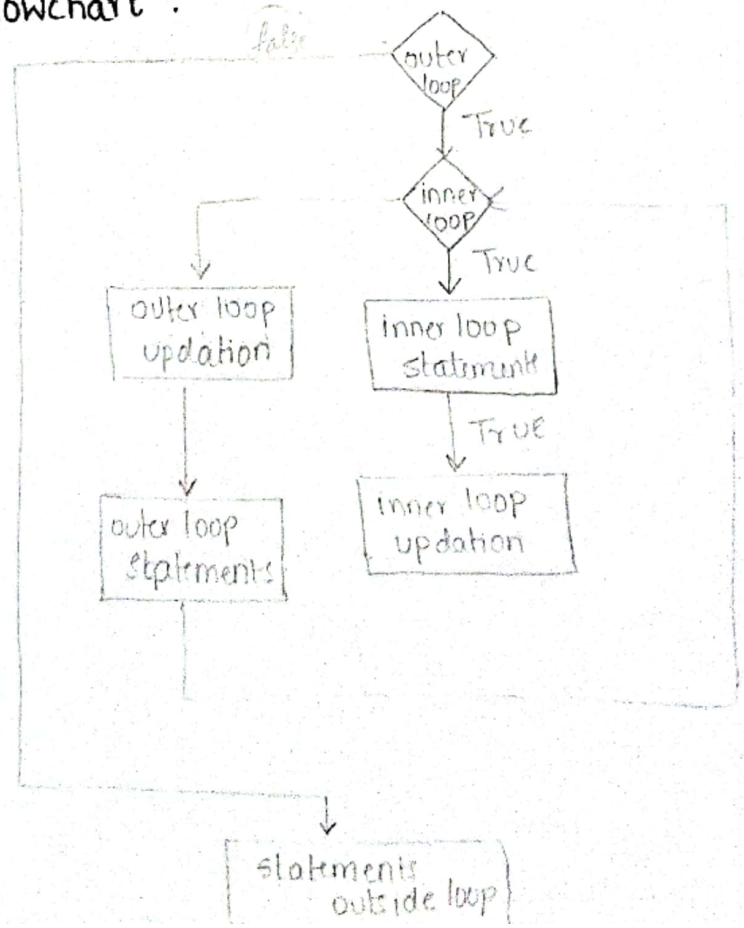
(i) Inner Loop
(ii) Outer Loop

Syntax:

```
for (initialization ; test condition ; updation)
{
for (initialization; test condition; updation)
{
}
}
```



Flowchart:



Scanned with OKEN Scanner

→

```
#include <stdio.h>
int main()
{
int i, j, n;
printf("Enter the value of n");
scanf("%d", &n);
for(i=1; i<=n; i++)
{
for(j=1; j<=5; j++)
{
printf("*");
}
printf("\n");
}
}
```



Tracing

i = 1
n = 6
1 <= 6 (T)
j = 1 2 3 4 5 6
1 <= 5 (T)
2 <= 5 (T)
3 <= 5 (T)
4 <= 5 (T)
5 <= 5 (T)
6 <= 6 (F)

i = 2
n = 6
2 <= 6 (T)
j = 1 2 3 4 5 6
1 <= 5 (T)
2 <= 5 (T)
3 <= 5 (T)
4 <= 5 (T)
5 <= 5 (T)
6 <= 5 (F)

i = 3
n = 6
3 <= 6 (T)
j = 1 2 3 4 5 6
1 <= 5 (T)
2 <= 5 (T)
3 <= 5 (T)
4 <= 5 (T)
5 <= 5 (T)
6 <= 5 (F)

i = 4
n = 6
4 <= 6 (T)
j = 1 2 3 4 5 6
1 <= 5 (T)
2 <= 5 (T)
3 <= 5 (T)
4 <= 5 (T)
5 <= 5 (T)
6 <= 5 (F)

i = 5
n = 6
5 <= 6 (T)
j = 1 2 3 4 5 6
1 <= 5 (T)
2 <= 5 (T)
3 <= 5 (T)
4 <= 5 (T)
5 <= 5 (T)
6 <= 5 (F)

i = 6
n = 6
6 <= 6 (T)
j = 1 2 3 4 5 6
1 <= 5 (T)
2 <= 5 (T)
3 <= 5 (T)
4 <= 5 (T)
5 <= 5 (T)
6 <= 5 (F)

i = 7
n = 6
7 <= 6 (F)

The control comes out of the loop.

Scanned with OKEN Scanner

```
#include<stdio.h>
int main()
{
int i,j,n;
printf("Enter the value of n");
scanf("%d",&n);
for(i=1; i<=n; i++)
{
for(j=1; j<=i; j++)
{
printf("%d",i);
}
printf("\n");
}
}
```

Tracing

i = 1	i = 2	i = 3	i = 4	i = 5
n = 4	n = 4	n = 4	n = 4	n = 4
1 <= 4 (T)	2 <= 4 (T)	3 <= 4 (T)	4 <= 4 (T)	5 <= 4 (F)
j = 1 2 (F)	j = 1 2 3	j = 1 2 3 4	j = 1 2 3 4 5	
1 <= 1 (T)	1 <= 2 (T)	1 <= 3 (T)	1 <= 4 (T)	The control comes out of the loop
2 <= 1 (F)	2 <= 2 (T)	2 <= 3 (T)	2 <= 4 (T)	
	3 <= 2 (F)	3 <= 3 (T)	3 <= 4 (T)	
		4 <= 3 (F)	4 <= 4 (T)	
			5 <= 4 (F)	

Output

```
1
2 2
3 3 3
4 4 4 4
```

Scanned with OKEN Scanner

* Jump Statements

→ C language supports 3 jump statements:

(i) goto
(ii) break
(iii) continue

1) goto

→ It is an unconditional jump statement which is used to move from one point to another point in the program.

```
#include <stdio.h>
int main()
{
  int i;
for (i=1, i<=5; i++);
{
printf ("\t i= %d", i);
  if (i==5)
  goto VITS;
}
printf ("Hi");
VITS:
  printf ("\n This is a labelled statement");
}
```

o/p :- i=1 i=2 i=3 i=4 i=5

This is a labelled statement

Scanned with OKEN Scanner

2) break

→ It is used to come out of the current loop

→ It terminates the loop body immediately & passes control to the next statement after the loop.

```
#include <stdio.h>
int main()
{
int i;
for (i=1; i<=10; i++);
{
if (i==5)
{
Printf ("%d\n", i);
break;
}
}
}
```

O/P :- i = 1 i = 2 i = 3 i = 4 i = 5

3] Continue

```
#include <stdio.h>
int main()
{
int i;
for (i=1; i<=10; i++)
{
if (i==5)
continue;
printf ("\n %d", i);
}
}
```

Scanned with OKEN Scanner

* Errors

→ Syntax Errors
→ Runtime Errors
→ Logical Errors
→ Linked Errors
→ Sematic Errors

Scanned with OKEN Scanner